UC San Diego
JACOBS SCHOOL OF ENGINEERING

DERCONNECT
TESTING FACILITY

Introduction to Optimization

Developer Charlie Frank, Sonali Caffrey

Advisors Phuong Truong, Keaton Chia, Jan Kleissl

Last Updated September 2, 2025

Website Outreach Website

Documentation Intro to Optimization

Learning Objectives

- **Define** optimization and explain its importance in energy systems
- **Identify** key optimization applications in DERConnect, including:
  - Battery scheduling
  - EV smart charging
  - Load shifting and demand response
- **Analyze** how optimization improves system efficiency, cost-effectiveness, and grid reliability
- **Apply** key concepts from DERConnect, including:
  - Microgrid management
  - Distributed control strategies

Standard Systems vs Optimal Systems

Standard Systems:

- Operate on **fixed**, pre-set schedules
- Often waste energy through overuse or underuse
- Tend to have higher energy costs
- Use **one-size-fits-all** control strategies
- Lack coordination between devices
- Respond slowly or not at all to grid changes

Optimal Systems:

- Make smart, **real-time** decisions based on data
- Use energy efficiently across all resources
- Lower operational and energy costs
- **Customize** responses to current conditions
- Coordinate multiple devices for best overall performance
- Adjust quickly and automatically to grid changes

What is the Goal of Optimization?

To make smart decisions that improve **efficiency**, **reliability**, and **cost-effectiveness**.

In DERConnect this means:

- Using the **least amount of energy** to get the job done
- Charging or discharging **batteries at the right time**
- Reducing energy use when prices are high or the grid is stressed
- Making sure **solar, EVs, and storage** work together smoothly
- Keeping power flowing **even during outages or cyber threats**

Quick Quiz

Standard Systems vs Optimal Systems

How do optimized systems handle multiple devices using energy resources

A. Devices take as much energy as they want, whenever they want

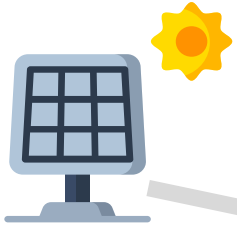
B. Smart controls coordinate when devices are used for best performance

C. Smart controls randomly shut devices on and off to reduce power usage

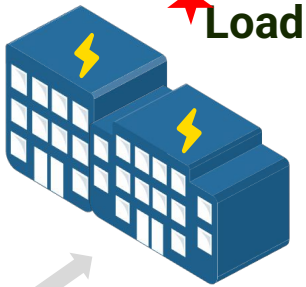
Introduction to Project in a Box Optimization

Load Profiles

Photovoltaic Power (PV)/solar power



Load



Main grid

Battery Energy Storage System (BESS)

The load and PV system each have daily energy profiles, allowing us to visualize their energy output and consumption over time

Load Profiles and Their Importance

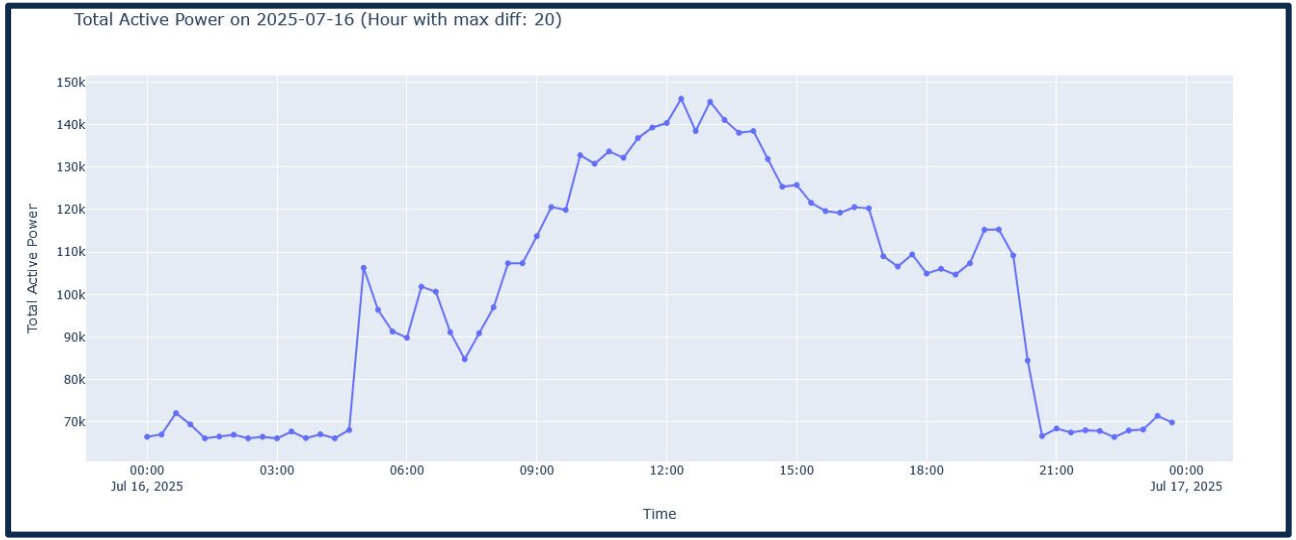
What is a Load Profile?

- A load profile is a graph showing **how electricity demand changes over time** (hourly, daily, seasonal)
- It tells us when energy use is high (peaks) and when it is low

Why It Matters in Optimization

- Helps match solar generation with demand patterns

  Identifies when batteries are most useful (charging in low demand, discharging in peaks)
- **Reduces costs** by planning around time-of-use rates
- Ensures system reliability by **predicting demand spike**



Here’s an example of the Student Services building load profile—take note of how the demand rises and falls over the course of the day

Battery Energy Storage System (BESS)

Q: What is a Battery Energy Storage System (BESS)?

- A large-scale rechargeable battery system that stores electricity for later use.

Q: Why is BESS important?

- Balances energy supply and demand.
- Improves grid stability and reliability

Q: How does BESS support renewable energy?

- Stores excess power from solar and wind for use when production is low.

Q: How does it relate to a load profile?

- A BESS can store energy during **low-demand (off-peak)** times and release it during **high-demand (peak)** times, smoothing out the load profile and improving grid efficiency.

Quick Quiz

Battery Energy Storage System (BESS)

Which of the following is NOT a purpose of a BESS?

a) Balancing energy supply and demand

b) Improving grid reliability

c) Generating fossil fuels

What is the Project in a Box Optimization

“Project in a Box” is a **self-contained learning module**:

- You download one zip file
- Inside: data, code, instructions, and utilities
- Goal: let students run a complete project with minimal setup

This project’s box = a **microgrid optimization simulator**.

In this simulation we are using **Google Colab**, a free cloud-based platform that allows us to write and run Python code in a web browser without installing anything on our own computer.

Let's Get Started!

Step 1: Upload the Project Files

Run this block:

```
from google.colab import files
uploaded = files.upload()
```

Here is a link to the Colab:
https://drive.google.com/file/d/1QQpTTPLdouSDcYUP3wZ9BGnPxaXJpFU9/view?usp=drive_link

A button appears → select **derconnect-project-in-a-box-P4.zip**
This gives Colab access to all project code and data

Step 2: Extract the Zip

We need to unpack the project so Python can use the files:

```
[ ] import zipfile

    with zipfile.ZipFile("derconnect-project-in-a-box-P4.zip", "r") as zip_ref:
        zip_ref.extractall("derconnect")

    %cd derconnect
```

Now Colab is “inside” the project folder

Step 3: Import Core Libraries

- **cvxpy** → optimization math
- **numpy** → numerical calculations
- **pandas** → data tables
- **plotly** → interactive graphs

These tools allow simulation, data handling, and visualization.

Quick Quiz

Project in a Box

Which Python library is used for optimization math?

a) pandas

b) plotly

c) numpy

d) cvxpy

Step 3: Install Requirements (cont.)

This optimizer uses a few specific libraries and files to work, run these two sections to ensure they are downloaded to your Colab session

```
[ ] !pip install -r /content/derconnect/derconnect-project-in-a-box-2-optimization/requirements.txt
```

```
[ ] from dotenv import load_dotenv
    load_dotenv(dotenv_path="/content/derconnect/derconnect-project-in-a-box-2-optimization/Software_P4/mvp1_temp/.env")
    import cvxpy as cp
    import numpy as np
    import pandas as pd
    import os
    import plotly.graph_objects as go
    import time
```

Step 4: Load Environment Variables

- Settings (like case study start days, file paths, and battery values) are stored in a **.env** file.
- Loading this ensures consistent configurations across runs

Step 4: Load Environment Variables (cont.)

The utility functions do the bulk of the work in this optimizer, run this section to ensure they are also loaded to the colab session

```
[ ] %cd /content/derconnect/derconnect-project-in-a-box-2-optimization/Software_P4/mvp1_temp
    from utilities.optimize_microgrid import optimize_microgrid
    from utilities.sim_results import sim_results
    from utilities.plotting import plotting
    from utilities.welcome import welcome
```

Step 5: Import Utility Functions

The project organizes helpers into separate files:

- **optimize_microgrid** → runs the optimization
- **sim_results** → builds results tables
- **plotting** → makes visual graphs
- **welcome** → asks user for input

This keeps code clean and modular

Step 6: Understanding Inputs

The optimizer asks for:

- **Rate model**: electricity pricing scheme
- **Case study**: which dataset/scenario to simulate
- **Battery size (BESS)**: Small, Medium, or Large

These inputs change the outcome of the optimization

Step 7: Running the Optimizer Code

The program uses your choices and environment values (solar data, demand data, battery specs) to simulate the microgrid

It runs **two scenarios**:

1. With a battery
2. Without a battery

This allows comparison of costs and performance

Quick Quiz

Why does the optimizer run both with and without a battery?

A) To test if the code runs faster

B) To compare costs and performance

C) To check if solar energy is available

D) To install requirements again

Step 8: Running the Optimizer Code Part 1

```
counter = 0
while True:
  # Welcome and input
  welcome_choices = welcome(counter=counter)
  rate_model = welcome_choices['rate_model']
  case_id = welcome_choices['case_id']
  bess_size = welcome_choices['bess_size'].strip().upper()
  counter = welcome_choices['counter']

  solar_start_day = int(os.getenv(f"CASE{case_id}_SOLAR_START_DAY"))
  demand_start_day = int(os.getenv(f"CASE{case_id}_DEMAND_START_DAY"))
  num_days = int(os.getenv(f"CASE{case_id}_NUM_DAYS"))
  output_file = str(os.getenv(f"CASE{case_id}_OUTPUT_FILE"))

  bess_capacity = float(os.getenv(f"BESS_{bess_size}_CAPACITY"))
  charge_rate = float(os.getenv(f"BESS_{bess_size}_CHARGE_RATE"))
  discharge_rate = float(os.getenv(f"BESS_{bess_size}_DISCHARGE_RATE"))
  efficiency = float(os.getenv(f"BESS_{bess_size}_EFFICIENCY"))

  %cd /content/derconnect/derconnect-project-in-a-box-2-optimization
  # Run simulation
  full_results = optimize_microgrid(solar_start_day= solar_start_day,
                                    demand_start_day= demand_start_day,
                                     num_days= num_days, rate_model = rate_model, bess_capacity = bess_capacity, charge_rate = charge_rate,
                                     discharge_rate = discharge_rate, efficiency = efficiency, enable_battery=True)
  nobess_results = optimize_microgrid(solar_start_day=solar_start_day,
                                      demand_start_day= demand_start_day,
                                       num_days= num_days,rate_model = rate_model, bess_capacity = bess_capacity, charge_rate = charge_rate,
                                       discharge_rate = discharge_rate, efficiency = efficiency, enable_battery=False)
```

Step 8: Running the Optimizer Code Part 1 continued

- The program asks you to choose a **pricing plan**, a **case study**, and a **battery size**
- Based on your choices, it looks up the right **solar and demand data** for that scenario
- It also gets the details of the battery you picked (like how much it can store and how fast it can charge)
- Then it runs the simulation **two times**: once with the battery included, and once without it
- Finally, it compares the results so you can see how much the battery changes costs and performance.

Step 8: Running the Optimizer Code Part 2

```
# Run simulation
full_results = optimize_microgrid(solar_start_day= solar_start_day,
                                  demand_start_day= demand_start_day,
                                   num_days= num_days, rate_model = rate_model, bess_capacity = bess_capacity, charge_rate = charge_rate,
                                   discharge_rate = discharge_rate, efficiency = efficiency, enable_battery=True)
nobess_results = optimize_microgrid(solar_start_day=solar_start_day,
                                    demand_start_day= demand_start_day,
                                     num_days= num_days,rate_model = rate_model, bess_capacity = bess_capacity, charge_rate = charge_rate,
                                     discharge_rate = discharge_rate, efficiency = efficiency, enable_battery=False)

# Output table with hourly results and basic grid and cost info
results_sim = sim_results(full_results=full_results,
                          nobess_results=nobess_results,
                          case_id = case_id, bess_size = bess_size, num_days=num_days,
                        output_file=output_file)

# Plotting results
goplot = plotting(full_results=full_results, nobess_results=nobess_results, num_days = num_days)

repeat = input("\nWould you like to run another simulation? (yes/no): ").strip().lower()
if repeat != "yes":
    print("\nThanks for exploring the Optimization Simulator. Goodbye!\n")
    break
```

Step 8: Running the Optimizer Code Part 2

Here is a summary of the code displayed in Part 2:

The program **runs the simulation** using all the data and battery settings you chose

- First, it simulates **with the battery**
- Then, it simulates the **same scenario without the battery**

It then **creates a results table** showing hourly energy usage, costs, and how the grid performed

Next, it **draws interactive graphs** so you can visualize energy flow, battery use, and cost savings
Finally, it asks if you want to **run another simulation**

- If yes, the process starts again
- If no, the program ends with a goodbye message

Quick Quiz

Which feature allows you to test multiple scenarios without restarting?

A) Optimization math

B) Interactive graphs

C) Simulation loop

D) Zip extraction

What We Learned

Here is a summary:

- How to set up and run a complete project in **Google Colab**
- What **optimization** means in the context of a microgrid
- How to compare **with-battery vs. without-battery** scenarios
- How to interpret results using **tables and interactive graphs**
- Why optimization is important for **cost savings and energy efficiency**

Real-World Impact

How optimization runs in reality:

Microgrid optimization mirrors real energy challenges

Utilities and communities use similar tools to:

- Lower electricity costs
- Improve renewable energy use
- Enhance reliability during peak demand

Skills learned here apply to **sustainability, engineering, and data science** fields

Final Thoughts

- Optimization isn’t just coding, it’s about **making better decisions with data**
- Microgrids show how small choices (like battery size) create big cost impacts
- This project demonstrates the value of **combining programming, math, and sustainability**
- Keep experimenting with new inputs to see how the system behaves!